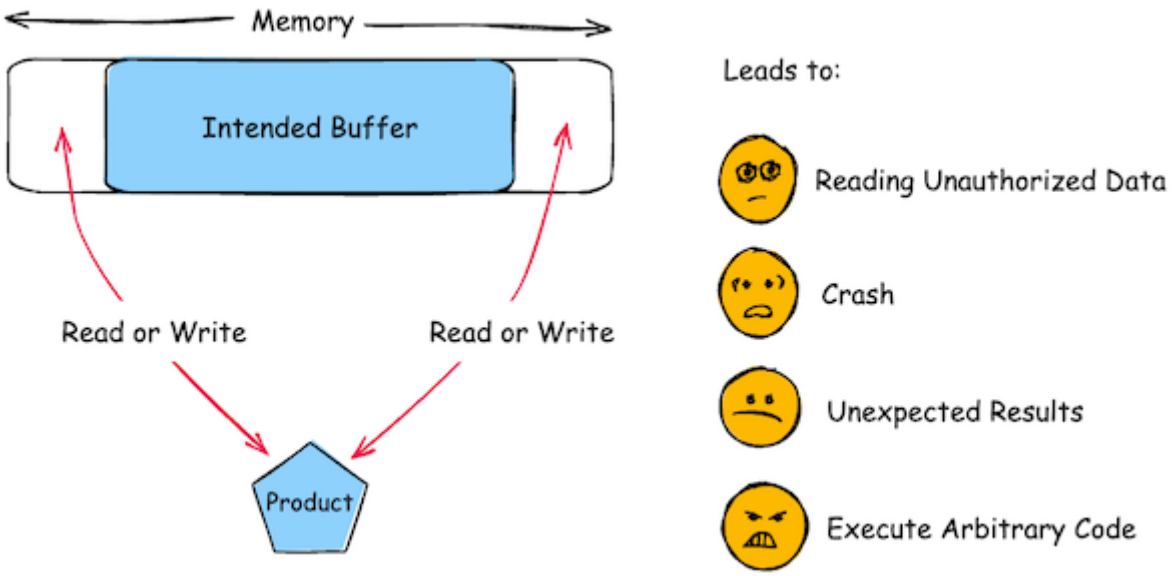
Introduction to Memory Vulnerabilities

A 30-Year-Old Problem

- CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer
- Improper coding and bounds checking, mainly in C/C++
- Attacker can take control of program executing code of its own
- **First major exploit**: Morris Worm (1988)
- **661 CVE-listed vulnerabilities in 2017**



Some terminology

- buffer overflow: read/write past the end of a buffer
- buffer overrun: alternate term for **buffer overflow**
- memory leak: every allocated block of memory should be properly freed when no longer needed
- invalid access: programs must not read from or write to memory locations they do not own
- memory safety: programs ensure that they access memory in valid, predictable ways, preventing common errors that can lead to crashes, data corruption, or security vulnerabilities.

Buffer Overflows: Still Dangerous

- **Aleph One**: Smashing the Stack for Fun and Profit, article in Phrack magazine, 1996
- **Vulnerability of the decade** (2000s)
- Most network penetration vulnerabilities (now overtaken by web app attacks)
- Total control if program is privileged

A Flurry of Attacks

- **1988:** Morris Worm, 1988 (10% of the Internet affected).
- **2001:** CodeRed (300,000 machines infected in 14 hours).
- **2002:** SQL Slammer ($1.2 billion loss).

- **2008:** Win32 Conficker ($9 billion loss, 10 million hosts).
- **2014:** Paper Unleashing MAYHEM on Binary Code.

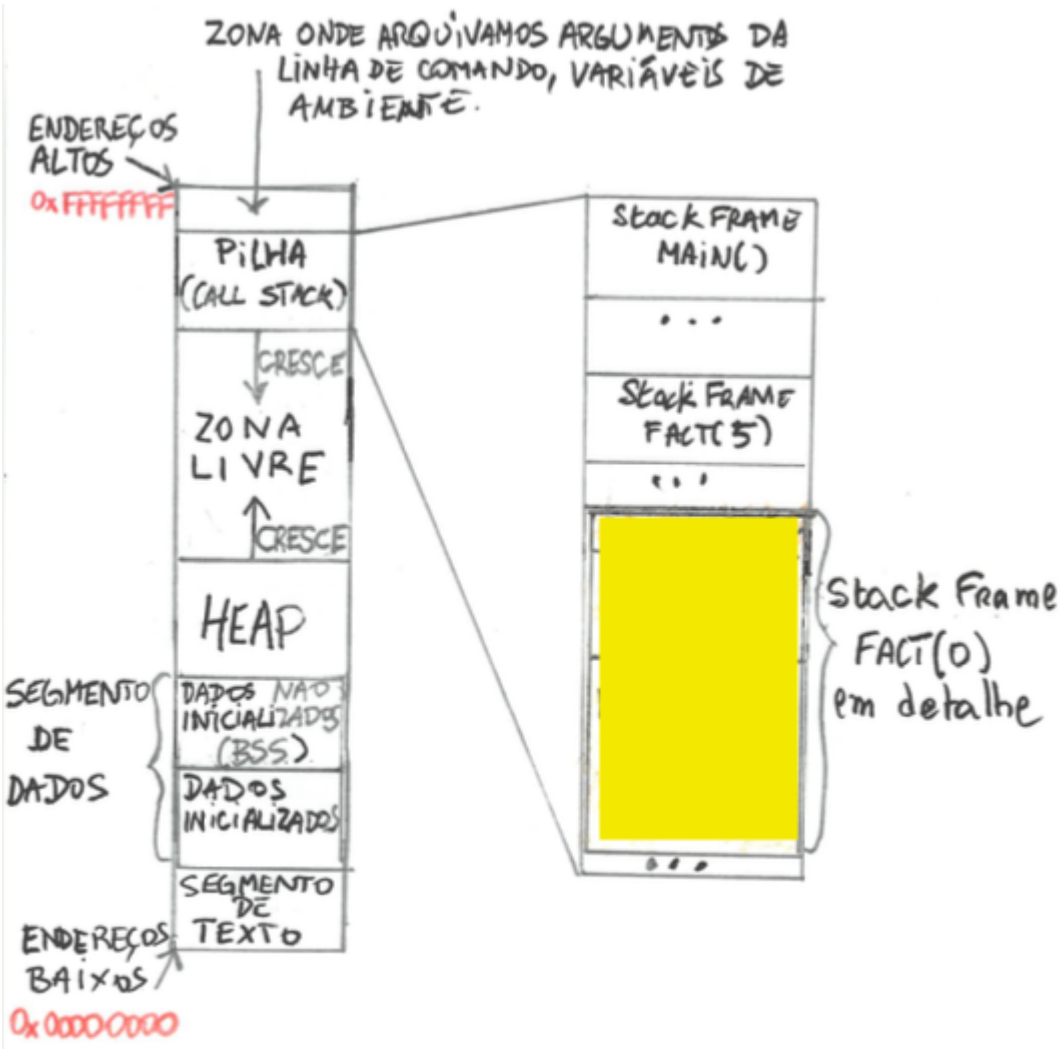
Classic Buffer Overflow in C

Unchecked String Bounds

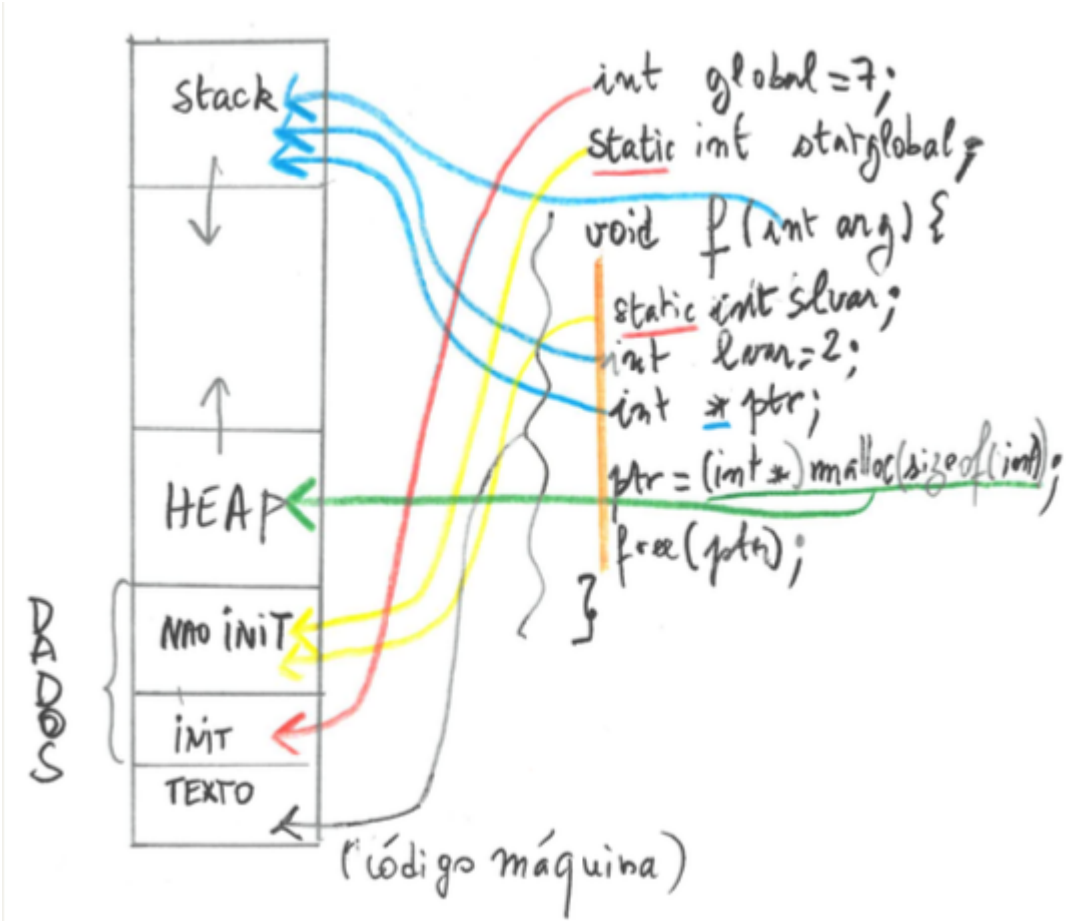
```c
int func(char *str) {
  char buffer[12];
  int variable_a;
  strcpy(buffer, str);  // Potential overflow
}
```

- String of unknown length strcopied into `12-char` buffer
- Copying a string of unknown length into a fixed-size buffer.
- **Risk**: Overwriting adjacent memory, including the return address.

Memory Layout in C (part 1)



Memory Layout in C (part 2)

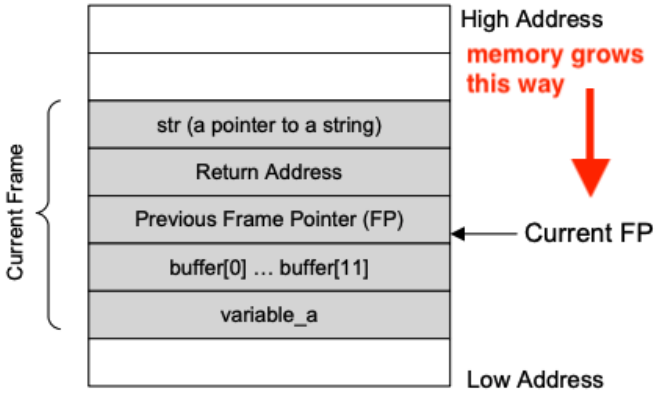


Memory Layout in C (part 3)

```
void func (char *str) {
  char buffer[12];
  int  variable_a;
  strcpy (buffer, str);
}

Int main() {
  char *str = "I am greater than 12 bytes";
  func (str);
}
```

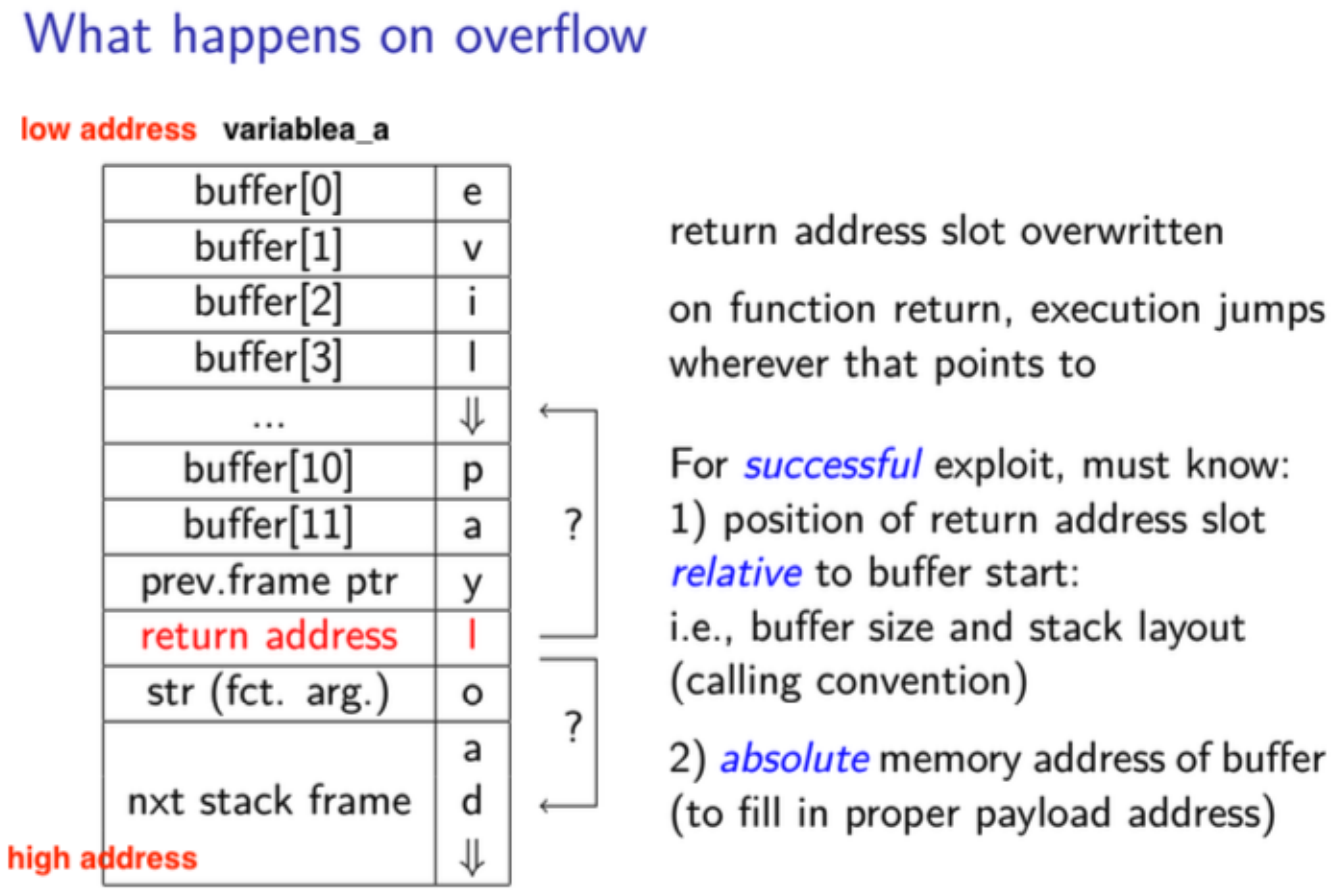
(a) A code example



(b) Active Stack Frame in func()

- Buffer Overflow Vulnerability Lab
- Memory Video
- **x86 (32-bit, SEED-style calling convention)**

Detailed Stack Frame



Buffer Overflow Vulnerability Lab

Attack 1: Overwrite the Return Address with a Fixed Address

```
void vulnerable(char *input) {
  char buffer[12];
  strcpy(buffer, input);
}
```

Attack payload

```
[12 bytes of junk][4 bytes of junk][4 bytes: return address]
```

```
[12 bytes of junk]      → fills the buffer (buf[12])
[4 bytes of junk]       → overwrites saved EBP
[4 bytes: address of system()] → overwrites return address
```

EBP = Extended Base Point = Previous Frame Pointer

Attack 2: Inject Shellcode into the Buffer, then Return to It

- to place malicious **shellcode** (e.g., code to launch `/bin/sh`) inside the `buffer`.
- to overwrite the return address so that when the function returns, it jumps into the buffer,executing the **shellcode**.

```
void vulnerable(char *input) {
  char buffer[12];
  strcpy(buffer, input);
}
```

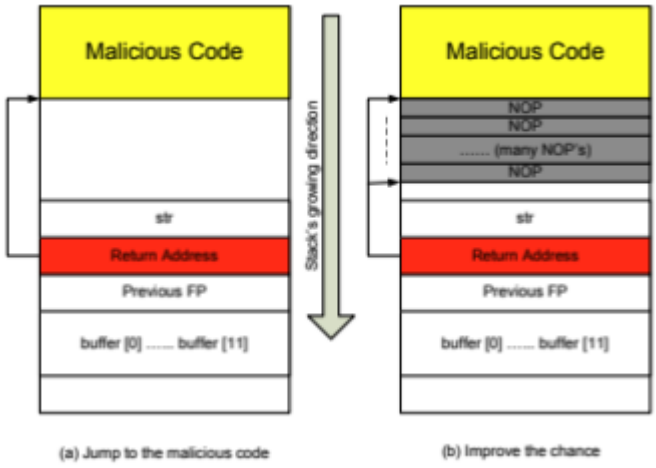
📦 Payload Structure

```
[ NOP sled          ]
[ Shellcode         ]
[ Padding           ]
[ Overwrite EBP     ]
[ Return address → somewhere in NOP sled ]
```

Part	Size	Purpose
`NOP sled` (`\x90`)	~16–100+ B	Sequence of NOPs to slide into the shellcode. Makes jumping easier.
`Shellcode`	~24–100 B	Machine code that performs the malicious action (e.g., spawns a shell).
`Padding`	Variable	Fills remaining buffer space to reach saved EBP.
`Saved EBP`	4 bytes	Overwritten value; not used in this attack (can be junk).
`Return address`	4 bytes	Overwritten with an address pointing into the NOP sled or shellcode.

Using NOP Sled

- Technique used in Attack 2



Format String Vulnerability

`fgets` Vulnerability

```c
int main(void) {
  char s[80];
  if(fgets(s, sizeof(s), stdin))
    printf(s);
}
```

🟦 x86 (32-bit, SEED-style calling convention)

```
Stack grows downward (high → low addresses)

+-----------------------------+ ← High Address
| Argument 3                  |  ← pushed by caller
| Argument 2                  |
| Argument 1                  |
| Return Address              |  ← pushed by `call`
| Saved Frame Pointer (EBP)   |
| Local Variables             |
+-----------------------------+ ← Low Address
```

🟢 AArch64 (64-bit ARM Calling Convention)

- **ARM64 Architecture**, Mac mini (with Apple Silicon: M1, M2, and M3)

```
↑ High Address
──────────────────────────────────+
| Local Variables                 | ← allocated by compiler
| Saved Frame Pointer (EBP)       |
| Return Address                  | ← pushed by `call`
──────────────────────────────────+
↓ Low Address

Registers used for arguments:
x0 → Argument 1
x1 → Argument 2
x2 → Argument 3
```

- **Return Addresses**
  - for an **exploit** we are interested in the return address from `main` to `_start` since this address lives longer in the stack than the return address from `printf` to `main`

```
_start → main → printf
```

Insecure Program Input

Always Limit Input!

C11 standard removed function ~~gets~~: did not limit size read
⇒ it is impossible to use ~~gets~~ safely
⇒ buffer overflow, memory corruption, security vulnerabilities

Use: `char *fgets(char *s, int size, FILE *stream);`

Reads up to and including newline `\n`, max. `size-1` characters, stores line in array `s`, adds `'\0'` at the end.

NEVER use ~~`%s: scanf("%s",...)`~~. Leads to buffer overflow.

MUST give max. string length in format!

```
char str[30];
if (scanf("%29s", str) != 1) { /* handle error */ }
else { /* word (up to first whitespace) is in s */ }
```

Always Check for successful (correct) input!

Reading the desired data might not succeed for two reasons:
- system: no more data (end-of-file), read error, etc.
- user: data not in needed format (illegal char, not number, etc.)

A function can report both a *result* and an *error code* as follows:

1. *expand result datatype* to include error code
   `getchar()` : `unsigned char` converted to `int`,
   or `EOF (-1)` which is different from any `unsigned char`
2. return type may have a special *invalid/error value*
   `fgets` returns address where the line was read (first argument)
   or `NULL` (invalid pointer value) when nothing read
3. return *error code* and *store result* at given pointer
   `scanf` *returns no. of items read* (can be 0, or `EOF` at end-of-input)
   takes as arguments *addresses* where it should place read data

Understanding EOF

Input functions only detect EOF if trying to read *past* the end
⇒ testing `feof(somefile)` can be misleading

e.g., read sequence of whitespace-separated numbers
if no more input follows, `feof(stdin)` true after read
if char follows (Enter, space, etc.) `feof(stdin)` still false
(but there may be no other number left)

⇒ can't and should not use `feof(...)` as test in the read loop

Process Input While Correct

Checking for end-of-input explicitly is rarely needed.
The point of processing is to *read data*
⇒ thus we must check that data was read successfully:

`while (`*read successful*`)` *use data*

On exit from loop, if `feof(stdin)`, input is finished
else input does not match format ⇒ read next char(s) and report

DO NOT write code of the form

```
while (!feof(stdin))
    scanf("%d", &n);
```

After last good read (number), end-of-input is not yet reached
unless no more separators (whitespace, incl. newline) after it
⇒ next read will not succeed, but is not checked
If read is checked (as it *MUST* be), testing `EOF` is not needed:

```
while (scanf("%d", &n) == 1)
  // process n
```

Checking Bounds While Filling an Array

Often, we have to fill an array up to some stopping condition:
read from input upto a given character (period, \n, etc)
copy from another string or array

Arrays must not be written beyond their length!

Loop should **first** *test array is not full!*

```
for (int i = 0; i < len; ++i) { // limit to array size
   tab[i] = ...;       // assign with value read
   if (normal stopping condition) break/return;
}
// here we can test if maximal length reached
// and report if needed
```

Error Handling

MUST check return code of *every* I/O function

Failure modes may not be obvious

`fclose(f)` : all done, can still have error
no room to flush to disk; USB stick removed; HW failure
⇒ must report, perhaps chance to recover

Even printing to `stdout` could fail (redirected, no more space)

Meaningful error messages: errno / perror

Error codes
global variable `int errno` declared in `errno.h`
contains code of last error in a library function
(illegal operation, file not found, not enough memory, etc.)

Careful: use `errno` only if sure that there was an error
for safety reset before calling function that may fail

Function `void perror(const char *s)` from `stdio.h`
prints user message `s`, a colon `:` and then the error description
(same as given by `char *strerror(int errnum)` from `string.h`)

String to num: use functions w/ error reporting

Converting strings to numbers `int n = atoi(s);` returns 0 on error, but also for `"0"`

Avoid. Use only when string known to be good.

```
int n; char s[] = "  -102  56 42";
if (sscanf(s, "%d", &n) == 1) ... //number OK
```

but we don't know where processing of string stopped

also does not signal overflow (if number too large)

```
long int strtol(const char *nptr, char **endptr, int base);
```

assigns to `*endptr` the address of first unprocessed char

`char *end; long n = strtol(s, &end, 10);` base 10 or other

also `strtoul` for `unsigned long`, `strtod` for base 10 `double`

set `errno` to `ERANGE` on overflow

What is the Root of Evil

C is *low-level*

No (safe) notion of memory object

Can create (almost) arbitrary pointers

Can't really pass an array to a function

pointer passed instead

address carries no length information

Must pass array length as separate parameter

but programmer responsible for passing correct value

even for heap-allocated chunk, length is available to library, but not checked on use

Unsafe Functions and Replacements

`strcpy`

`strncpy` – but was really designed for fixed-length strings

`strcat` seldom a logical reason to use

`strncat` careful: can write n+1 bytes

`sprintf`

`snprintf`